Summary of Pre-Training 2/3

Jamal Hassunizadeh (jamalvh), Ajay Kumar (kumaraj), Tejas Maire (tmaire), Allison Okimoto (aokimoto)

TrainVerify: Equivalence-Based Verification for Distributed LLM Training

Problem and Motivation

With recent studies establishing that scaling a deep neural network generally yields better model performance, there is a growing need to train larger models. Training these larger models, especially modern large language models (LLMs), requires extensive resources and computational workload across clusters with thousands of GPUs across weeks and months, known as distributed training. Distributed training is highly complex, requiring careful partitioning of operators and tensors, correctly scheduling to appropriate devices, proper insertion of communication operators, and precise coordination of execution order. These complexities must also preserve the original model's semantics, given that training occurs under varying configurations and parallelization approaches, like data, tensor, and pipeline parallelism.

While distributed training is essential for scalability, this high degree of complexity makes it inherently error-prone. Studies from three distributed training systems, *MegatronLM*, *DeepSpeed*, and *nnScaler*, reveal that bugs specific to parallelization arise in almost every part of the workflow. Problematically, these bugs are often silent, which can make the training appear sound, but results in incorrect model parameters due to subtle errors in gradient updates and improper scaling of model states. Not to mention, these parallelization bugs often evade traditional testing mechanisms. Floating-point computation, variations in the order of operations, and mixed-precision training make it challenging to distinguish normal numerical noise from actual errors. Additionally, simulating large-scale training at full-scale on a single-device for comparison is infeasible, and leveling down to smaller-scale tests can miss bugs that only appear in full-scale training.

Given that training LLMs operates on an extremely large-scale across a significant time period, the presence of bugs can result in a waste of computational resources, financial loss, and low productivity for developers. Moreover, incorrectly trained models can have serious consequences when deployed in real-world environments where model accuracy is important. Because of this, it is a necessity to provide formal correctness guarantees for parallelization to completely eliminate parallelization bugs in training LLMs.

Related Works

Prior research related to TrainVerify can be grouped into neural network equivalence, neural network verification, and correctness of deep learning (DL) training.

Eleftheriadis et al. encodes neural networks into SMT constraints, a similar manner to TrainVerify, but targets approximate equivalence for knowledge distillation and is constrained to two-layer feed-forward neural networks. Other neural network optimizers, like TASO and PET, accelerate neural networks by applying more efficient substitutions or partially equivalent transformations, but only reason about a small number of operators. Unlike TrainVerify, prior systems fail to scale to models with billions of parameters that modern LLMs require.

Existing neural network verification tools aim to address whether a trained neural network satisfies certain requirements and input-output specifications, by encoding the concrete weights and evaluating specific cases. In contrast, TrainVerify verifies equivalence by generalizing across all possible inputs by operating on symbolic tensors.

Runtime monitoring systems are used to evaluate correctness of DL training. TrainCheck, for example, checks invariants from previously validated pipelines to analyze end-to-end correctness, but can overlook errors due to numerical drift and uncaptured metrics. TrainVerify complements this by providing symbolic verification and formal correctness guarantees for parallelization, rather than relying solely on runtime observations.

Overall, related work in this field addresses components of the overall problem but lacks a scalable, end-to-end correctness guarantee of distributed LLM training that is TrainVerify.

Solution Overview

TrainVerify is a formal verification system that ensures the correctness of distributed training execution plans for neural networks. Instead of trying to verify the entire distributed training stack, which would involve various components like compilers, kernel libraries and communication runtimes, the authors of the paper focus on verifying the execution plan itself. Their insight is that if the distributed execution plan is mathematically equivalent to the logical definition of the model, then the correctness of the distributed training can be confirmed without having to build the entire system from the ground up.

The essence of the TrainVerify system is the concept of Parallelization Equivalence (PE), which basically says that for every possible input, the distributed execution must produce identical outputs to those generated by the logical model. To make this happen, TrainVerify extracts two data flow graphs, or DFGs: one representing the logical model and one representing the distributed execution plan. These graphs essentially capture operators as nodes and tensors as edges, which allows the system to reason over how computations are partitioned and coordinated between devices.

TrainVerify then converts both DFGs into symbolic representations by replacing tensor values with symbolic variables and rewriting operators into mathematical expressions. This abstraction allows for reasoning over algebraic semantics rather than floating-point outputs, which strives to block out numerical noise. Then, using lineage

metadata, TrainVerify aligns tensors in the distributed graph with their counterparts in the logical graph, ensuring that the system compares corresponding computations correctly.

To scale verification to large language models with billions of parameters, TrainVerify uses two methods: staged verification and shape reduction. Staged verification partitions the computation graph into smaller subgraphs, or stages, that can be verified independently but also in parallel. This process allows TrainVerify to establish end-to-end equivalence between the logical and distributed models, while having control over the verification problem. Next, shape reduction strives to minimize tensor sizes while maintaining their functional and structural properties. Since many deep learning operators follow a single-instruction-multiple-data (SIMD) pattern, ensuring correctness on a reduced tensor implies correctness on the full tensor.

Finally, TrainVerify uses an SMT solver to construct and verify formulas, which represent the parallelization equivalence across all inputs. If the equivalence does not hold, the solver has the potential to produce counterexamples that help developers identify bugs. Through this comprehensive methodology, TrainVerify provides a scalable and practical approach to formally verify distributed training plans in a simpler manner.

Limitations

Although TrainVerify provides strong verification for distributed training, it has a couple of limitations as well. One key constraint is its reliance on the logical model as ground-truth: if the model itself contains errors, TrainVerify will verify equivalence against an incorrect reference, which could essentially just end up propagating mistakes rather than detecting them. In this sense, TrainVerify preserves consistency between implementations but does not validate whether the logical model is correct itself.

TrainVerify also depends on symbolic modeling of operators, and this requires rewriting deep learning operations into mathematical definitions to be solved by a solver. While the authors show that this is possible through dozens of different operators, expanding support to more complex/custom operators may introduce engineering overhead and make it hard to immediately apply them to evolving distributed training frameworks.

Additionally, TrainVerify only focuses specifically on verifying execution plans and using parallelization equivalence logic, but it does not attempt to verify lower-level components like communication libraries, GPU kernels, or runtime implementations. Any bugs that originate from these layers could still affect the correctness of distributed training even if the execution plan has been verified. Accomplishing end-to-end correctness would require integrating multiple verification approaches.

Lastly, although staged verification and shape reduction improve scalability, the verification process still introduces overhead. For extremely large or highly customized training pipelines solver complexity could present itself as a bottleneck, which could affect the system's ability to keep up in development environments.

Future Research Directions

One potential future research direction is to extend verification beyond execution plans in order to include additional layers of the training stack, such as communication runtimes, kernel implementations, and specific hardware-level behaviors. Having this extra level of granularity into the TrainVerify framework could move the field closer towards fully verified distributed training.

Another direction to look into involves establishing automation in operator modeling. Developing standardized mathematical specifications for deep learning operators could reduce the manual effort that's needed to extend the TrainVerify process to new architectures and custom operations.

Future work could also explore continuous verification approaches. Instead of only verifying execution plans before training begins, systems could dynamically verify modifications to pipelines as they adapt to newer hardware configurations, parallelization strategies, and optimization techniques. This would help maintain correctness in environments, as distributed training environments could frequently change.

Lastly, the process of shape reduction could potentially imply opportunities in broader applications of symbolic reasoning within the machine learning space. Reduction techniques from TrainVerify could assist in other forms of verification, debugging, and optimization in distributed systems, and this can further strengthen reliability and scalability in future AI models.

Oobleck: Resilient Distributed Training of Large Models Using Pipeline Templates

Problem and Motivation

Deep neural network (DNN) models are continuing to become larger, and recent advances have led to increases in model size and parameters they are being trained on. Recently, training these large models is done with hybrid-parallel distributed training, which is a combination of model and data parallelism. However, because of the size and duration of training, it also significantly increases the likelihood of hardware and node failures.

The probability of failures in training increases because of synchronous distributed training. When a device fails, all other devices are stopped until the failed device recovers. This leads to underutilization. The current fault-tolerant mechanisms fall into two categories, checkpointing and redundant computation. Checkpointing is where training progress is periodically saved. This does not increase significant overhead, but it does have a recovery time when failures occur. Redundant computation is where each pipeline stage is computed redundantly into two nodes. When one of the nodes fails, the other computes the backward and forward passes for that node. This avoids having to restart, but introduces computational and memory overhead. Neither of these approaches provides strong fault-tolerance guarantees for hybrid-parallel training, where model states are distributed

across devices. Current systems struggle to maintain high throughput and underutilization during failures.

The main problem is that hybrid-parallel training needs a solution that provides fault-tolerance while maintaining throughput. The problem is important for making large-model training reliable and practical at a large scale. This would reduce the problems with underutilization and improve the scalability of distributed training.

Related Works

Prior research related to Oobleck can be grouped into elastic training systems, distributed training on spot instances, and optimizations for large model training.

Elastic training frameworks such as Horovod Elastic and TorchElastic address failures by restarting training after recovery, allowing the number of workers to change dynamically. Other systems, including CoDDL, Aryl, and Pollux, focus on improving resource efficiency and scheduling decisions in shared, elastic environments. However, these approaches are primarily designed for data-parallel training and assume relatively small models that fit on a single GPU, which limits their applicability to large-scale hybrid-parallel training.

Distributed training with spot instances has explored resilience under frequent preemptions. Varuna uses hybrid parallelism and reconfigures the training setup after failures, but recovery requires restarting from checkpoints. Bamboo introduces redundant computation within pipeline parallelism to tolerate failures without restarting, at the cost of additional computation and memory overhead. In contrast, Oobleck avoids both frequent restarts and persistent redundancy, achieving higher throughput across a wide range of model sizes and failure scenarios.

Finally, many systems have proposed techniques to improve the efficiency of large model training, including better parallelism strategies, memory management, and communication optimizations. While these approaches improve scalability and performance, they typically do not provide fault tolerance by default and are therefore orthogonal to Oobleck's focus on resilient distributed training.

Solution Overview

Oobleck is a fault tolerant hybrid parallel training framework. It uses the pipeline templates to guarantee fault tolerance and achieve high throughput. Oobleck's heterogeneous pipeline execution allows it to use all available GPUs, improving issues with underutilization.

Pipeline templates are specifications of how many nodes should be used by a pipeline, how many stages in a pipeline, and how model layers are mapped in stages to GPUs. Oobleck instantiates pipelines from these templates. All of the pipelines are logically equivalent and compute the same model. However, they are physically heterogenous, meaning they use an arbitrary number of nodes and different configurations. This decouples the pipeline planning from execution, which speeds up failure recovery.

Oobleck is able to create all possible configurations for pipelines to utilize all GPUs through its planning algorithm. The planning algorithm consists of template generation and instantiation planning. During template generation, the algorithm determines the number of pipeline templates and the number of nodes each template uses. This ensures that any number of remaining nodes can be utilized in the case of failure. Once the number of nodes in each pipeline template is determined, the algorithm finds the number of pipeline stages and then maps the nodes to the stages. It uses a divide-and-conquer algorithm to minimize the iteration time. During instantiation planning, the algorithm enumerates the templates and chooses the one with the highest estimated throughput. This algorithm makes sure that after any failure Oobleck can resume training with high GPU utilization.

Oobleck instantiates f + 1 pipeline replicas from the pipeline templates. This guarantees that when there is a failure, the system can tolerate up to f simultaneous failures without losing all the replicas of a pipeline stage. This is because in the worst case, f node failures would result in f pipelines failing. In a general case, Oobleck can tolerate more than f failures if one or more copy of the model states is retained across the pipelines. When a failure happens, Oobleck does not restart. Pipelines are reinstated using the templates to use all nodes. During this, the nodes share the model states and copy missing model states from each other. Once both of these things are done, the nodes continue training. If the cluster cannot hold f + 1 replicas, Oobleck stores the current progress so the user could resume from that checkpoint when the nodes can hold the replicas.

Limitations

One of the most significant limitations of Oobleck is its lack of adaptability, stemming from the fact that its pipeline layout is determined entirely before training begins and cannot be altered at runtime. In other words, Oobleck relies on pipeline templates that are generated once during an initial planning or configuration phase, and these templates remain fixed and immutable throughout the duration of the training job. While this upfront planning allows the system to quickly reconfigure in response to failures and maintain high throughput, it also inherently restricts its ability to adjust to changing or unforeseen circumstances. For example, variations in hardware across different nodes, fluctuations in system performance over time, or even shifts in the optimal batch size as training progresses are difficult for Oobleck to respond to dynamically. In practical deployments, where conditions often evolve in complex and unpredictable ways, this rigidity can result in suboptimal resource utilization and missed opportunities to optimize performance. This limitation underscores a fundamental design tradeoff in Oobleck: the system emphasizes fault tolerance and scalable throughput, but it does so at the cost of reduced operational flexibility during execution, which could have meaningful implications for certain practical workloads.

Future Research Directions

As stated, Oobleck currently relies on ahead-of-time pipeline planning, which inherently limits its flexibility and responsiveness to dynamic or unforeseen conditions during execution. In the future, it would be valuable to explore approaches that allow pipelines to be adapted and adjusted dynamically at runtime, in response to a variety of observed factors. These factors might include changes in hardware availability or variations in node performance, fluctuations in network bandwidth or latency between nodes, or gradual shifts in the optimal batch sizes as training progresses. Such adaptability could potentially be achieved through a combination of techniques, including continuous runtime profiling, more sophisticated adaptive scheduling strategies, or even reinforcement learning-based approaches that allow the system to learn and optimize pipeline configurations on the fly. The ultimate goal would be to enable Oobleck to maintain high throughput and robust fault tolerance, while also improving its ability to respond intelligently and efficiently to evolving execution conditions.

Summary of Class Discussion

TrainVerify and Oobleck differ in their approaches, in that TrainVerify utilizes proactive verification, whereas Oobleck is suited towards reactive recovery. Additionally, when it comes to logical equivalence, TrainVerify explicitly verifies execution graphs, and Oobleck assumes equivalence by construction.

When it comes to integrating the methods, a sequential approach from Superbench to Oobleck to TrainVerify is ideal. We can start by using Superbench to benchmark the nodes and detect slow/bad nodes. From there, the qualified nodes are passed to Oobleck, from which the pipeline templates are generated and instantiated. Finally, the execution plans from Oobleck are passed to TrainVerify to verify equivalence and eliminate parallel bugs.

<u>Q&A/Discussion</u>

- How would verification logic for TrainVerify change if GPUs of different machines were used for training?
    - Switching templates would not affect TrainVerify as long as both templates are verified
- Did the development of these models reduce manual optimization (spending less time making sure the code is correct)
    - TrainVerify still requires manual specification, like for the operators, but it's hard to do optimizations by hand so it does make it faster
- Is the main benefit of TrainVerify that it finds bugs faster, or are there additional benefits? Did the paper mention finding bugs that were not reported before?
    - Yes, it is the main benefit. The bugs are usually nuanced and hard to find. It's a better process than manually finding the bugs. The paper did not mention catching bugs that were unknown.

- Could Oobleck be updated to automatically add new computers to the mix while it's already running training?
    - Yes. Oobleck considers both node failure and node addition.
- Who are the users of these tools?
    - The users are distributed training system (ex: Megatron) developers, infrastructure developers, and not regular developers, as these tools require formal specifications about how the computation looks like, which is a challenge.
- How can failure be detected and when is it time to take action to fix these failures?
    - Failure in distributed systems is very hard to detect, and reliably detecting is itself a research problem. Oobleck assumes fail stop cases, which are easy to reason about, and they default assume failure by injecting failures.
- Will TrainVerify and other forms of verification make developers lazy (since it does a lot of error finding itself)?
    - It can happen, but typically even if these kinds of systems didn't exist, people try to write very carefully. These systems are often written assuming that people have put in a lot of effort but that there will still be some human error and mistakes that makes it hard for humans to reason about, because we don't have that much memory and patience to try out many different ways. In general, the main problem with all of these verifications systems and why they're still not widely used is that first you have to formally specify what the computation should look like; that part is often still manual and very difficult for anyone to do it.
- How is the logical definition of the code being written?
    - The logical version of the code is the code that you write with PyTorch; you would write down the code for the whole transformer, and you're not too worried about your device's capability. The parallelized version is the version that we implement things like data parallelism, tensor parallelism, and we run that in a cluster.
- What is the purpose of the lineage data structure?
    - The lineage data structure helps TrainVerify identify, given a group of parallel tensors, which ones do they correspond to in the logical model. When it finds the source in the logical model, we can combine those back together and compare the results.